What is Prolog?

- **Invented** early seventies by Alain Colmerauer in France and Robert Kowalski in Britain.
- Prolog = **Programming in Logic**.
- Prolog is a **declarative programming language** (unlike most common programming languages, where programmer must specify in detail how to solve a problem).
- In a declarative language:
  - the programmer specifies what the problem is, and a goal to be achieved
  - the Prolog system works out how to achieve it
- A program is **partly like a database** but much more powerful since we can also have general rules to infer new facts!
- A **Prolog interpreter** can follow these **facts/rules** and **answer queries by sophisticated search**.

SWI is derived from Sociaal-Wetenschappelijke Informatica ("Social Science Informatics")

 # Start

$swipl

[$swipl -s file.pl] [$swipl -s file.pl -t goal]

 # Exit:

halt.

ctrl-D

 # help

help(help).

apropos(help).

 # load a file called file.pl

consult('file.pl').

consult(file).

['file.pl'].

[file].

 # Create & Edit

edit(file('file.pl')).

 # Edit

edit('file.pl').

 # Command-line Editor

[user].

Two built-in predicates that can be used to load clauses into the Prolog database:

1. consult/1
2. reconsult/1.

consult('file1.pl')

```
dog(fido).
dog(rover).
dog(jane).
dog(tom).
dog(fred).
cat(mary).
cat(harry).
```

file2.pl

```
dog(henry).
dog(fido).
cat(bill).
cat(steve).
```

consult('file2.pl')

```
dog(fido).
dog(rover).
dog(jane).
dog(tom).
dog(fred).
dog(henry).
dog(fido).
cat(mary).
cat(harry).
cat(bill).
cat(steve).
```

reconsult('file2.pl')

```
dog(henry).
dog(fido).
cat(bill).
cat(steve).
```

```
# trace
trace. % on
notrace. % off

# spy a predicate "pred" % enter into trace when a particualr predicate is called:
spy(pred/n). % of n arguments
spy(pred). % of any argument
nospy(pred/n). % off
nospy(pred). % off

# Useful command while in trace mode:
<enter>: move one step forward
l: leap causes the program to resume until the next spy point, or until the program is done
s: skip the current subprogram (with trace off, and the trace resumes when that subprogram is done)
a: (abort:) exit the program and stop tracing

# the output of trace consists of following information:
% <Call/Redo/Exit/Fail:> <Recursion depth> <the name of the current goal> <creep>
% Call: prolog is trying to evaluate the goal
% Redo: .. to find another goal
% Exit: the goal has succeeded
% Fail: there is no other solution

# debug
debug. % on
nodebug. % off
```

What's Prolog? Let's do a test drive!

Here is a simple Prolog program saved in a file named ***family.pl***

```
male(albert).                          %a fact stating albert is a male
male(edward).
female(alice).                         %a fact stating alice is a female
female(victoria).
parent(albert,edward).                 %a fact: albert is parent of edward
parent(victoria,edward).
father(X,Y) :-                         %a rule: X is father of Y if X if a male parent of Y
        parent(X,Y), male(X).  %body of above rule, can be on same line.
mother(X,Y) :- parent(X,Y), female(X).  %a similar rule for X being mother of Y
```

- A fact/rule (statement) ends with "." and white space ignored
- read :- after rule head as "if". Read comma in body as "and"
- Comment a line with % or use /* */ for multi-line comments
- Ok, how do we run this program? What does it do?

But what can we do with our program?

We can ask **queries** *after loading our program*:

```
male(albert).     %this is our family.pl program
male(edward).
female(alice).
female(victoria).
parent(albert,edward).
parent(victoria,edward).
father(X,Y):- parent(X,Y), male(X).
mother(X,Y):- parent(X,Y), female(X).
```

?-male(albert).

Yes. *%the above was true*

?-male(victoria).

No. *%the above was false*

?-male(mycat).

No.

?-male(X). *%X is a **variable**, we are asking "**who** is male?"*

X=albert; *%right! now type **semicolon** to ask for more answers.*

X=edward;

No *%no more answers*

?-father(F,C). *%F & C are variables, we are asking "**who** is father of **whom**"*

F=albert, C=edward; ***%this is awesome! How did Prolog get this?***

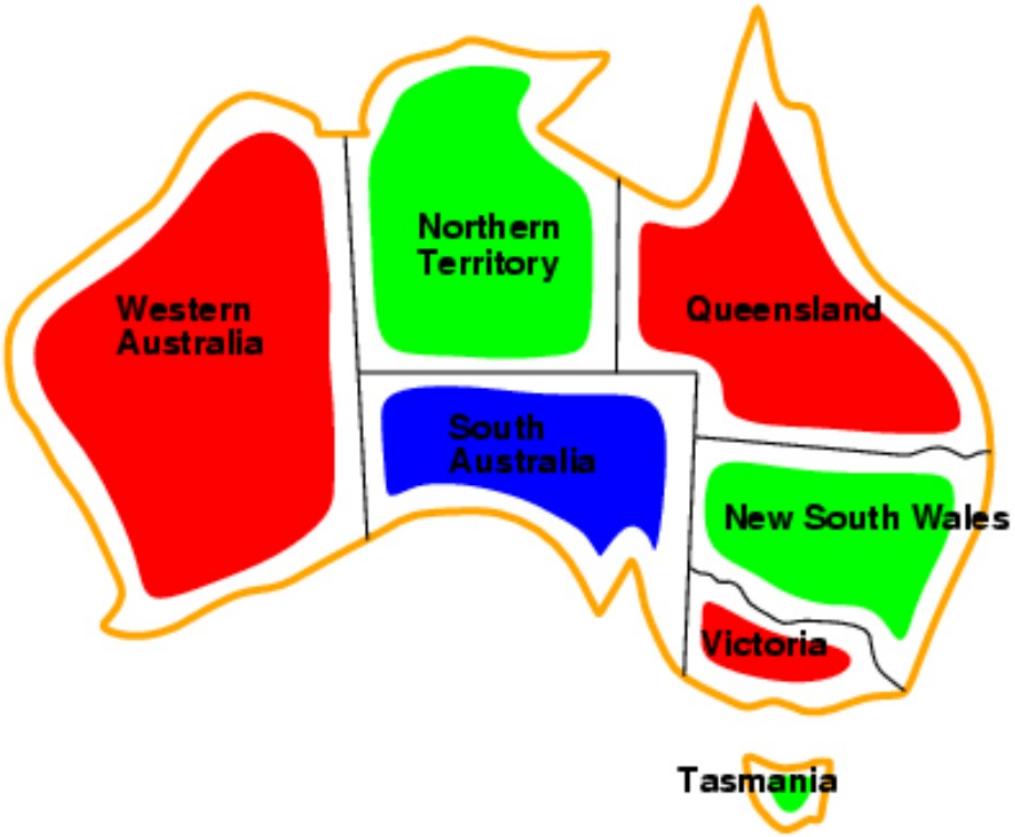
No

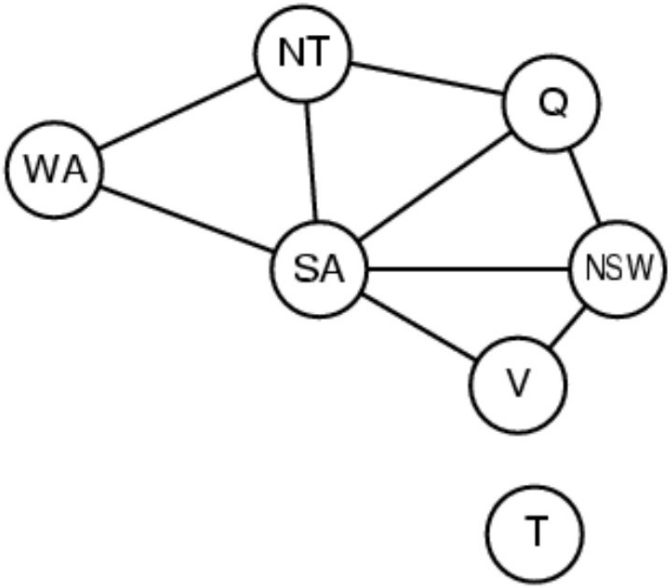
?- grandFather(X,Y):-

```
likes(alice,bob).
likes(bob,carol).
likes(james,mary).
likes(mary,james).

love_compatible(X, Y) :- likes(X, Y), likes (Y, X).
```

?- love_jealousy(X,Y):-

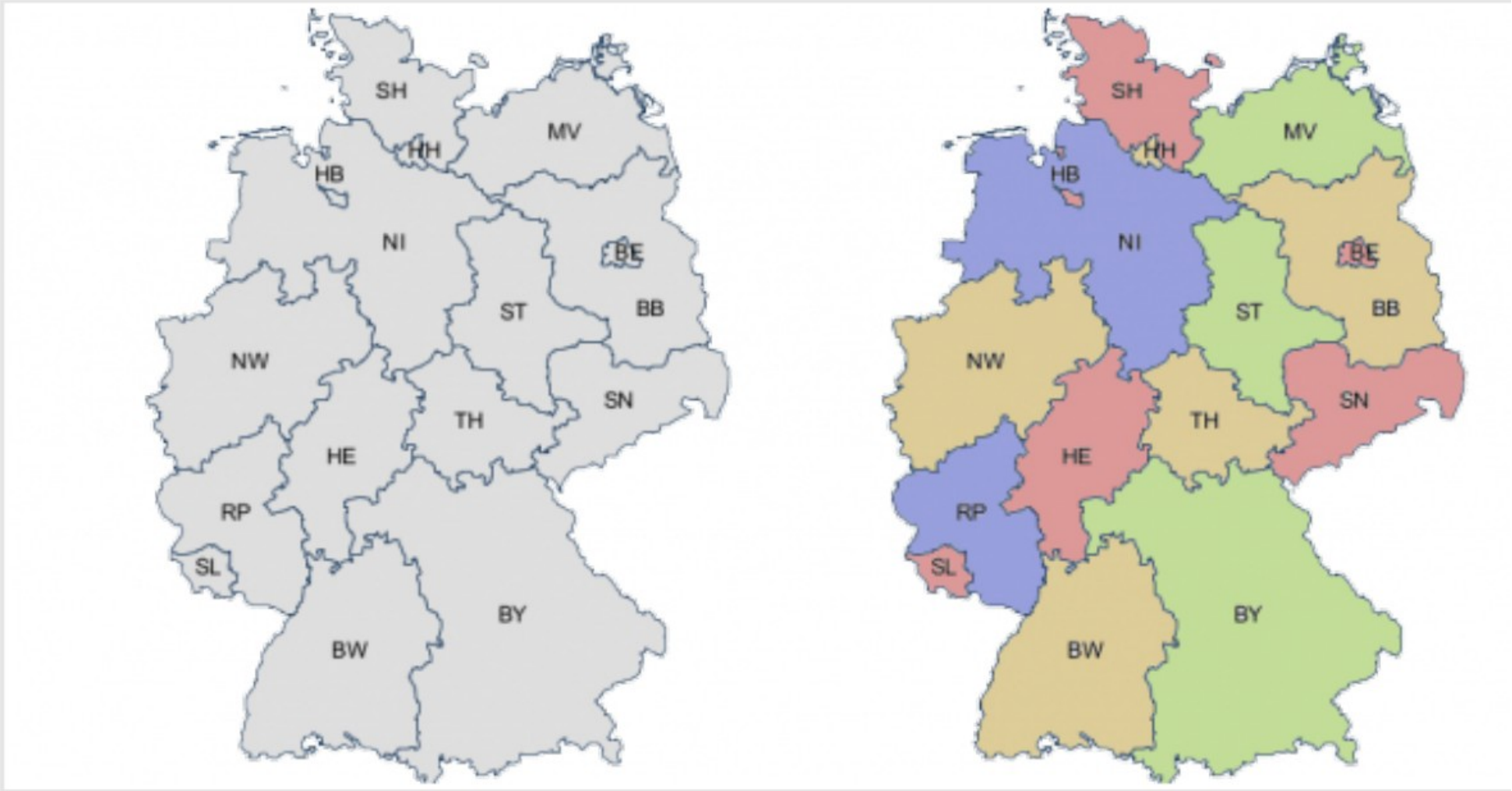





```
color(red).
color(green).
color(blue).
color(yellow).
```

```
neighbor(StateAColor, StateBColor) :- color(StateAColor),
    color(StateBColor),
    StateAColor \= StateBColor. /* \= is the not equal opera
```

```
australia(WA,NT,SA,Qu,NSW,Vic,Tas):-
    neighbor(WA,NT),
    neighbor(WA,SA),
    neighbor(NT,SA), ...
```



The map coloring problem applied to Germany and it’s states. Click on it for a larger

Operator	Meaning	Syntax
>	greater than	`Expression1 > Expression2`
<	less than	`Expression1 < Expression2`
>=	greater than or equal to	`Expression1 >= Expression2`
=<	less than or equal to	`Expression1 =< Expression2`
=:=	equal to	`Expression1 =:= Expression2`
=\=	not equal to	`Expression1 =\= Expression2`

`?- X =:= 3+2.` `ERROR: =:=/2: Arguments are not sufficiently instantiated`	`X` is not currently bound, so can't be evaluated.
`?- X = 3+2.` `X = 3+2.`	`=` doesn't evaluate, so `X` is bound to `3+2`.
`?- X is 3+2.` `X = 5.`	`is` *does* evaluate its right-hand side.
`?- 4+1 is 3+2.` `false.`	`3+2` is evaluated to `5`. `4+1` is not evaluated. So `4+1` is different from `5`.
`?- 4+1=3+2.` `false.`	Neither side is evaluated by `=`. The two expressions are different.
`?- 4+1 =:= 3+2.` `true.`	Both sides are evaluated by `=:=`

```
?- write('Hello World'),nl,write('Welcome to Prolog'),nl.

   Hello World
   Welcome to Prolog
   yes
```

```
?- read(X).
: jim.
X = jim
```

```
?- put(97),nl.
a
yes

?- put(122),nl.
z
yes
```

```
?- get0(N).
: a
N = 97

?- get0(N).
: Z
N = 90
```

All men are mortal.
Socrates is a man.

Hence, Socrates is mortal.

```
mortal(X) :- man(X).
man(socrates).
```

```
?- mortal(socrates).
Yes
```

```
?- p(X, 2, 2) = p(1, Y, X).
No


?- p(_, 2, 2) = p(1, Y, _).
Y = 2
Yes
```

```
public int factorial(int n) {
  if (n == 1) {
    return 1;                        // base case
  } else {
    return factorial(n-1) * n;  // recursion step
  }
}
```

```
factorial(1, 1).          % base case

factorial(N, Result) :-  % recursion step
  N > 1,
  N1 is N - 1,
  factorial(N1, Result1),
  Result is Result1 * N.
```

```
get_answer(Ans):-
   write('Enter answer to question'),nl,
   repeat,write('answer yes or no'),read(Ans),
   valid(Ans),write('Answer is '),write(Ans),nl.
valid(yes). valid(no).
```

```
?- gcd(57, 27, X).
X = 3
Yes

?- gcd(1, 30, X).
X = 1
Yes

?- gcd(56, 28, X).
X = 28
Yes
```

```
dog(fido).
dog(fred).
dog(jonathan).

alldogs:-dog(X),write(X),write(' is a dog'),nl,fail.
alldogs.
```

?- alldogs.
fido is a dog
fred is a dog
jonathan is a dog
yes

```
?- [1, 2, 3, 4, 5] = [Head | Tail].
Head = 1
Tail = [2, 3, 4, 5]
Yes
```

```
?- [quod, licet, jovi, non, licet, bovi] = [_, X | _].
X = licet
Yes
```

```
len([], 0).                 % base case

len([_ | Tail], N) :-  % recursion step
  len(Tail, N1),
  N is N1 + 1.
```

```
concat_lists([], List, List).

concat_lists([Elem | List1], List2, [Elem | List3]) :-
  concat_lists(List1, List2, List3).
```

```
?- concat_lists([1, 2, 3], [d, e, f, g], X).
X = [1, 2, 3, d, e, f, g]
Yes
```

```
?- concat_lists(X, Y, [a, b, c, d]).

X = []
Y = [a, b, c, d] ;

X = [a]
Y = [b, c, d] ;

X = [a, b]
Y = [c, d] ;

X = [a, b, c]
Y = [d] ;

X = [a, b, c, d]
Y = [] ;

No
```

```
?- length([elephant, [], [1, 2, 3, 4]], Length).
Length = 3
Yes

?- member(dog, [elephant, horse, donkey, dog, monkey]).
Yes
```

`last/2`: This predicate succeeds, if its second argument matches the last element of the list given as the first argument of `last/2`.

`reverse/2`: This predicate can be used to reverse the order of elements in a list. The first argument has to be a (fully instantiated) list and the second one will be matched with the reversed list. Example:

```
?- reverse([1, 2, 3, 4, 5], X).
X = [5, 4, 3, 2, 1]
Yes
```

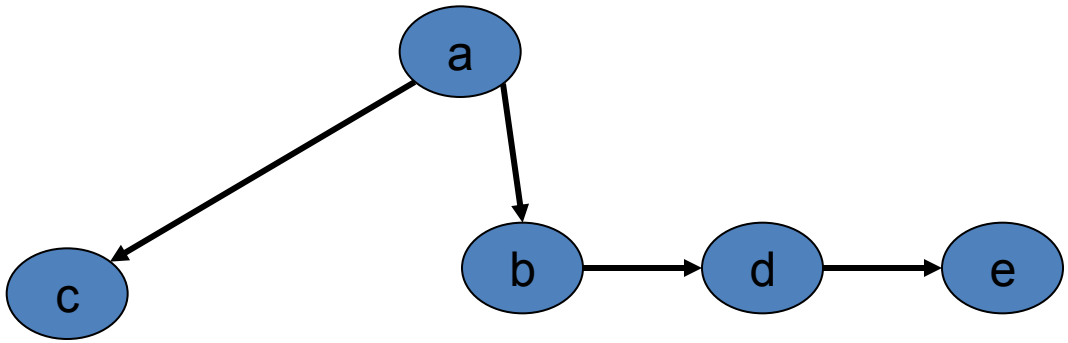
`select/3`: Given a list in the second argument and an element of that list in the first, this predicate will match the third argument with the remainder of that list. Example:

```
?- select(bird, [mouse, bird, jellyfish, zebra], X).
X = [mouse, jellyfish, zebra]
Yes
```

Another Example of Query Answering

- Route finding in in a directed acyclic graph:

edge(a,b).
edge(a,c).
edge(b,d).
edge(d,e).



path(X,Y) :- path(X,Z), edge(Z,Y).
path(X,Y) :- edge(X,Y).

- The above is problematic. Why?
- Here is the correct solution:

path(X,Y) :- edge(X,Y).
path(X,Y) :- edge(X,Z), path(Z,Y).

Hojjat Ghaderi and Fahiem Bacchus, University of Toronto

Search Tree for Path Predicate

Here is the program:

```
edge(a,b).
edge(a,c).
edge(b,d).
edge(d,e).

path(X,Y) :- edge(X,Y).
path(X,Y) :-  edge(X,Z), path(Z,Y).
```

Queries:

```
path(a,e).    [run SWI interactively to see this in action.]
path(c,R).
path(S,c).
```

What if the graph is undirected? Simple:

```
undirEdge(X,Y):- edge(X,Y).
undirEdge(X,Y):- edge(Y,X).
```

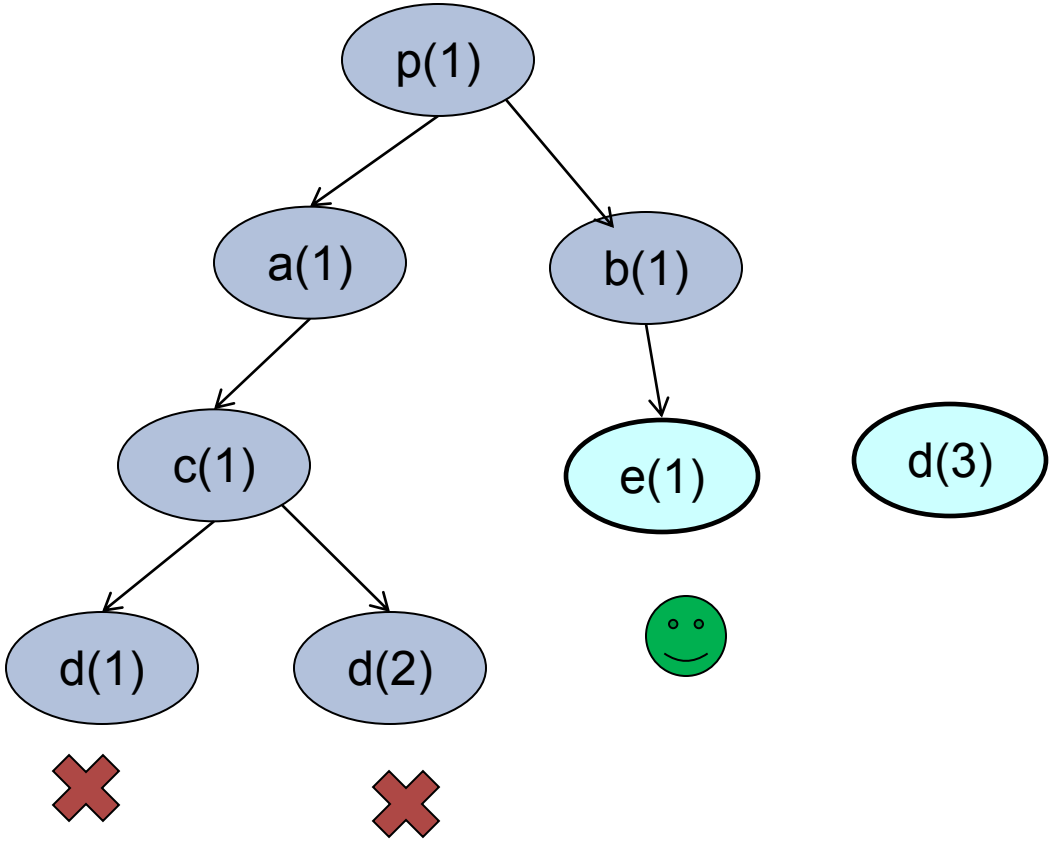
then replace *edge* predicate by *undirEdge* in the definition of path.

Hojjat Ghaderi and Fahiem Bacchus, University of Toronto

Operation

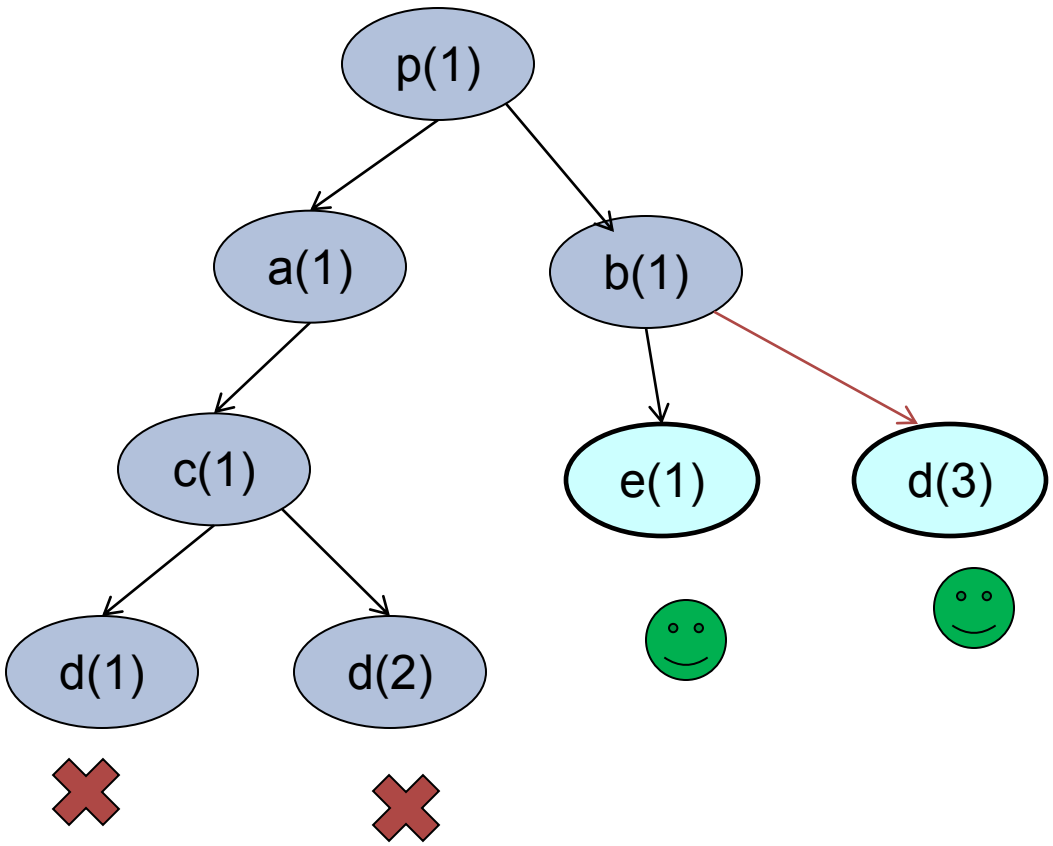
- Backtracking can occur at every stage as the query is processed.

- p(1) :- a(1).
  p(1) :- b(1).
  a(1) :- c(1).
  c(1) :- d(1).
  c(1) :- d(2).
  b(1) :- e(1).
  e(1).
  d(3).
- **Query**: p(1)



Operation

- With backtracking we can get more answers by using ";"
- p(1) :- a(1).
  p(1) :- b(1).
  a(1) :- c(1).
  c(1) :- d(1).
  c(1) :- d(2).
  b(1) :- e(1).
  b(1) :- d(3).
  e(1).
  d(3).
- **Query**: p(1)



CUT (!) Predicate

- Suppose a young prince wants to get married.
- In the old days he’d have saddled his horse to ride down to the valleys to find a young, beautiful, and intelligent girl.
- But, these days, times have changed, life in general is becoming much more complex – our prince is rather busy defending monarchy against democracy
- Fortunately, royal board of advisors consists of some psychologists and Prolog programmers in the country.

CUT (!) Predicate

- The board forms an executive committee to devise a Prolog program to automatise the prince’s quest for a bride.
- The task is to simulate as closely as possible the prince’s decision if he actually were to go out there and look for her by himself.

CUT (!) Predicate

- From the expert psychologists we gather the following information:
  - The prince is primarily looking for a beautiful girl. But, to be eligible for the job of a prince’s wife, she’d also have to be intelligent.
  - The prince is young and very romantic. Therefore, he will fall in love with the first beautiful girl he comes across, love her for ever, and never ever consider any other woman as a potential wife again. Even if he can’t marry that girl.

CUT (!) Predicate

- The MI5 provides the committee with a database of women of the appropriate age.
- The entries are ordered according to the order the prince would have met them on his ride through the country.

```
beautiful(claudia).
beautiful(sharon).
beautiful(denise).
...

intelligent(margaret).
intelligent(sharon).
...
```

```
beautiful(claudia).
beautiful(sharon).
beautiful(denise).
...

intelligent(margaret).
intelligent(sharon).
...

bride(Girl) :-
  beautiful(Girl),
  intelligent(Girl).
?- bride(X).
```

```
beautiful(claudia).
beautiful(sharon).
beautiful(denise).
...

intelligent(margaret).
intelligent(sharon).
...

bride(Girl) :-
  beautiful(Girl), !,
  intelligent(Girl).
?- bride(X).
No
```

```
larger(A,B,A):-A>B.

larger(A,B,B).
```

?- larger(8,6,X).
X = 8 ;
X = 6
?-

```
sumto(1,1).

sumto(N,S):-N1 is N-1,sumto(N1,S1),

    S is S1+N.
```

?- sumto(3,S).
S = 6

```
larger(A,B,A):-A>B.
larger(A,B,B).
```

```
larger(A,B,B):-A=<B.
```

?- larger(8,6,X).
X = 8 ;
X = 6
?-

```
sumto(1,1).
sumto(N,S):-N>1,N1 is N-1,sumto(N1,S1),S is
S1+N.
```

```
larger(A,B,A):-A>B,!.
larger(A,B,B).
```

```
sumto(1,1):-!.
sumto(N,S):-N1 is N-1,sumto(N1,S1),
   S is S1+N.
```

```
married(peter, lucy).
married(paul, mary).
married(bob, juliet).
married(harry, geraldine).

single(Person) :-



?- single(mary).
No

?- single(claudia).
Yes

?- single(X).
No
```

```
married(peter, lucy).
married(paul, mary).
married(bob, juliet).
married(harry, geraldine).

single(Person) :-
  \+ married(Person, _),
  \+ married(_, Person).

?- single(mary).
No

?- single(claudia).
Yes

?- single(X).
No
```

Lab Tasks

1. Implement a Prolog predicate occurrences/3 to count the number of occurrences of a given element in a given list.

```
?- occurrences(dog, [dog, frog, cat, dog, dog, tiger], N).
N = 3
Yes
```

2. Write a Prolog predicate divisors/2 to compute the list of all divisors for a given natural number.

```
?- divisors(30, X).
X = [1, 2, 3, 5, 6, 10, 15, 30]
Yes
```

** Make sure your program doesn't give any wrong alternative solutions and doesn't fall into an infinite loop when the user presses the semicolon key.